

MODULE 3.2: EISA, RISK, SECURITY, VULNERABILITY ASSESSMENT, PENETRATION TESTING

Contents

1. Enterprise Information Security Architecture (EISA).....	1
2. Risk Management Lifecycle	2
3. Security Policies & Frameworks	4
4. Network Lifecycle Security	6
5. Vulnerability Assessment & Penetration Testing	7
VAPT Phases.....	8
6. Penetration Testing Methodologies	8

Enterprise Information Security Architecture (EISA)

In enterprise environments, information systems are structured in **multiple layers**. Each layer must be secured individually and in coordination with others. This layered security design is key to protecting data, services, and infrastructure.

- 1. Business Layer:** This layer defines the **organizational goals, processes, and roles**. It focuses on aligning security with business objectives and ensuring that security policies support overall risk strategy. Example: Setting policies for secure online transactions in a banking application.
- 2. Data Layer:** This layer involves **storage, access, classification, and protection of data**. It ensures the confidentiality, integrity, and availability of business-critical information. Security controls include encryption, access control, backup strategies, and data masking. Example: Encrypting customer data in a healthcare database.
- 3. Application Layer:** This layer includes all **software applications and services** that users interact with. Application security focuses on protecting against code vulnerabilities, unauthorized access, and data exposure. Examples of controls: secure coding practices, input validation, API security, and access controls.
- 4. Network Layer:** The network layer handles **data transmission between devices, systems, and environments**. Security at this layer focuses on protecting data in transit and network infrastructure. Common controls include firewalls, VPNs, IDS/IPS, network segmentation, and secure protocols (e.g., HTTPS, SSH).
- 5. Physical Layer:** This refers to **physical security of hardware, devices, data centers, and facilities**. Controls include biometric access, CCTV, entry logs, fire suppression, and device locks. Even strong cyber defenses fail if an attacker can physically steal or tamper with servers.
- 6. Security Domains & Controls:** Security domains represent areas like identity management, endpoint protection, and network security. Controls can be administrative (policies), technical (encryption, firewalls), or physical (locks, surveillance). Examples:
 - **Data Security:** Protecting information (encryption, DLP).
 - **Application Security:** Securing software code and logic.

- **Network Security:** Protecting the perimeter and internal traffic.
- **Identity and Access Management (IAM):** Controlling who can access what (AAA).
- **Endpoint Security:** Protecting user devices (laptops, phones).

7. **Defense-in-Depth:** Defense-in-depth is a **security strategy that layers multiple defenses** at different points to protect systems. If one layer fails, others still stand to resist the attack. For example, an attacker might bypass the firewall but still face endpoint protection, access controls, and logging alerts. Key idea: **Never rely on a single control — use layers of protection.**

Layer	Description	Focus Area
Business	The top layer, defining the mission, goals, and high-level processes of the organization.	Policies, Governance, Compliance, Mission alignment.
Data	The information itself, including its structure, sensitivity, and lifecycle (creation, storage, use, destruction).	Confidentiality, Integrity, Availability (CIA) , Classification, Encryption.
Application	The software and services used to interact with the data and execute business logic.	Secure Coding, Authentication/Authorization (AAA), Web Application Firewalls (WAF).
Network	The communication infrastructure that connects devices, including routers, switches, and connections.	Firewalls , Network Segmentation (VLANs), Intrusion Detection/Prevention Systems (IDS/IPS).
Physical	The actual hardware, facilities, and environmental controls that house the IT equipment.	Locks, Guards, CCTV , Biometrics, Environmental controls (HVAC, fire suppression).

Risk Management Lifecycle

Risk management is the process of **identifying, analyzing, evaluating, and reducing risks** that may affect the confidentiality, integrity, or availability of systems and data.

1. **Risk Identification:** Sources of risk include system vulnerabilities, insider threats, malware, third-party dependencies, and natural disasters. Example: Unpatched software might lead to data breaches. **Discovering** the assets, threats, and vulnerabilities is very crucial. This involves:
 - **Assets:** What needs protecting (data, hardware, people, reputation).
 - **Threats:** What potential danger exists (malware, insider, natural disaster).
 - **Vulnerabilities:** Weaknesses in protection (unpatched software, weak policy).
2. **Risk Analysis:** Once identified, risks are analyzed to determine their **likelihood of occurring** and **potential impact** on the organization. Risks are usually categorized as **low, medium, or high**, based on factors like data sensitivity, exposure, and criticality of the system.
 - **Quantitative Analysis:** Assigns **numerical values** to risk in terms of potential monetary loss, likelihood, and impact. Example: Expected loss from a ransomware attack = ₹10,00,000 per incident × 20% chance = ₹2,00,000 risk.
 - **Qualitative Analysis:** Uses **descriptive categories** like high, medium, or low based on expert judgment, experience, or interviews. Example: An unprotected public-facing web server is “High Risk”. Both methods are often combined in real-world scenarios.

Aspect	Qualitative Method	Quantitative Method
Nature	Subjective, Judgment-based	Objective, Data-driven, Mathematical
Output	Descriptive rankings: High, Medium, Low	Numerical value: Monetary loss (\$ amount), Probability (%)
Technique	Risk Matrix, Expert Opinion, Brainstorming.	Cost/Benefit Analysis, Single Loss Expectancy (SLE), Annualized Loss Expectancy (ALE).
Advantage	Quick, cost-effective, easy to communicate.	Provides an objective, defensible cost for risk, enables ROI calculation for controls.

3. **Risk Mitigation:** This step involves applying **controls or strategies to reduce risk**. Mitigation methods include:

- Applying patches
- Implementing firewalls
- Changing configurations
- Enforcing strong access policies
- Employee training

Not all risks can be removed completely, but they can be minimized to an acceptable level.

Example Mitigation Actions

Strategy	Description	Example
Mitigate (Reduce)	Applying controls to lessen the likelihood or impact of the risk.	Patching a server vulnerability or implementing MFA.
Transfer	Shifting the financial burden of the risk to a third party.	Purchasing Cyber Insurance .
Avoid	Eliminating the risk entirely by choosing not to engage in the risky activity.	Discontinuing an outdated, vulnerable service.
Accept	Acknowledging the risk and its potential impact, often because the cost of mitigation is greater than the potential loss.	Documenting and accepting the low risk of a specific, rare power outage.

4. **Risk Monitoring and Review:** Once controls are applied, the organization continuously monitors them to ensure they are working. Threats change over time — new vulnerabilities appear, systems get updated, and attackers evolve. Regular reviews, audits, and security assessments help detect new risks and ensure controls remain effective.

5. **Continuous Improvement:** Based on monitoring results, policies and controls are improved or updated. If a risk becomes more severe or new threats arise, the organization re-evaluates and repeats the lifecycle. This stage ensures that risk management is **dynamic, adaptive, and aligned** with business and technology changes.

6. Example Mitigation Actions

- Enforce multi-factor authentication (MFA) for login security
- Regularly update and patch software
- Encrypt sensitive data at rest and in transit
- Segment networks to limit attacker movement
- Conduct regular vulnerability scans and audits
- Train employees on phishing and social engineering risks

Security Policies & Frameworks

A security policy is a **formal written document** that defines how an organization protects its information, systems, and resources. It outlines rules, expectations, and responsibilities for employees and IT teams. **Purpose**

- Establish consistent security practices
- Protect data, systems, and networks
- Reduce organizational risk
- Provide guidance during security incidents
- Ensure compliance with legal and regulatory standards
- Define acceptable and unacceptable behavior for users

Scope: The scope specifies **where and to whom** the policy applies — such as employees, contractors, vendors, cloud systems, networks, devices, or specific departments.

Standards, Procedures, Guidelines, and Policies

1. **Policies:** High-level statements describing *what* must be done. Example: “All sensitive data must be encrypted.”
2. **Standards:** Detailed rules that support policies and define *specific requirements*. Example: “Data must be encrypted using AES-256.”
3. **Procedures:** Step-by-step instructions explaining *how* to carry out a task. Example: Steps to set up disk encryption on a workstation.
4. **Guidelines:** Recommended best practices that are optional but helpful. Example: Recommendations for choosing strong passwords.

These four elements work together to ensure consistent and effective security implementation.

Document	Focus	Level of Detail	Enforcement
Policy	What and Why	High-level, strategic intent.	Mandatory (Senior Management directive).
Standard	What (Technology)	Prescriptive, mandatory rules that specify technical requirements (e.g., must use AES-256 encryption).	Mandatory (Support policy compliance).
Procedure	How	Detailed, step-by-step instructions for carrying out a specific task.	Mandatory (Ensures consistency and compliance).
Guideline	How (Best Practice)	Recommendations and best practices; advisory in nature.	Optional (Allows flexibility/judgment).

Organization-Wide, Issue-Specific, and System-Specific Policies

1. **Organization-Wide Policies:** Broad, top-level policies applicable to the entire organization. Example: Acceptable Use Policy for all employees.
2. **Issue-Specific Policies:** Focused on specific topics or risks. Example: Email usage policy, password policy, mobile device policy.
3. **System-Specific Policies:** Detailed rules for specific systems or technologies. Example: Database security policy, firewall configuration policy.

Policy Levels (High-Level and Low-Level)

1. **High-Level Policies:** Provide overall direction and goals. These are written for management and employees. Example: “The organization must ensure the confidentiality of customer data.”

2. **Low-Level Policies:** More technical and detailed, targeted at IT/security teams. Example: “Firewall must block all inbound traffic except ports 80 and 443.”

Types of Security Policies

1. **Security Testing & Audit Policy:** Defines how and when security testing such as vulnerability assessments, penetration tests, or audits will be conducted. It ensures systems are regularly evaluated for weaknesses.
2. **Incident Response Policy:** Contains guidelines for detecting, reporting, and responding to security incidents. It outlines team responsibilities, communication plans, and recovery steps.
3. **Digital Evidence Policy:** Provides rules for collecting, handling, storing, and preserving digital evidence. Ensures evidence is admissible in legal investigations and follows chain-of-custody requirements.
4. **Disaster Recovery & Business Continuity Policy:** Defines procedures for restoring IT systems and business operations after disasters. Focuses on recovery time objectives (RTO), backup planning, alternate sites, and continuity strategies.

Liability & Associated Risk

Liability refers to the legal responsibility for an action or omission that causes harm. In cybersecurity, this is a significant risk.

- **Associated Risk:** A security breach can lead to several types of financial and legal liability:
 - **Regulatory Fines:** Penalties for failing to comply with laws (e.g., GDPR, HIPAA).
 - **Civil Litigation:** Lawsuits from customers, employees, or business partners whose data was compromised.
 - **Contractual Penalties:** Fines for breaching security clauses in contracts with vendors or clients.
 - **Reputational Damage:** Loss of customer trust and market value (often the largest financial loss).
- **Mitigation:** Policies serve as **due diligence** documentation, demonstrating that the organization has taken reasonable steps to prevent harm, which can help limit liability in court.

Compliance vs Risk;

- **Compliance** means following laws, regulations, and standards (e.g., ISO 27001, PCI-DSS, GDPR).
- **Risk management** focuses on identifying and reducing real security threats. Sometimes organizations may be compliant but still vulnerable if risks are not addressed.

Role of Senior Management

Senior management is responsible for:

- Approving security policies
- Providing resources (budget, personnel, tools)
- Ensuring compliance and accountability
- Setting the organizational security culture
- Overseeing risk management decisions

Leadership involvement is crucial because strong security starts from the top.

Network Lifecycle Security

Network Lifecycle Security is the process of applying security controls and best practices throughout the entire existence of a network or system, from its initial conception to its final retirement. This ensures security is **integrated**, not just added as an afterthought. **Stages of the Network Lifecycle**

- 1. Initiation:** This is the **planning and requirements gathering** stage. Security begins by defining the purpose of the network and the security needs it must meet. **Key Activities:**
 - **Define Security Goals:** Determine the required level of **Confidentiality, Integrity, and Availability (CIA)** for the data the network will handle.
 - **High-Level Risk Assessment:** Identify potential threats and regulatory requirements.
 - **Develop Security Policy:** Establish the foundational **security policies** that will govern the network.
 - **Budgeting:** Allocate funds for security controls, personnel, and future audits.
- 2. Acquisition & Development:** In this stage, the network is designed, and components are purchased or built. Security focuses on **secure design** principles. **Key Activities:**
 - **Security Architecture Review:** Design the network with security in mind (e.g., **Defense-in-Depth**, network segmentation, secure zones).
 - **Secure Coding/Configuration:** If building applications, follow **Secure Software Development Life Cycle (SSDLC)** principles. If acquiring hardware, ensure vendors meet security standards.
 - **Component Selection:** Choose security-rated equipment (e.g., trusted firewalls, routers with strong authentication).
 - **Testing Plan:** Develop a plan for security testing (e.g., vulnerability scanning, penetration testing) before deployment.
- 3. Implementation:** The network components are configured, installed, and integrated into the environment. Security focuses on **secure deployment**. **Key Activities:**
 - **Secure Configuration:** Harden all devices, change default passwords, disable unnecessary services, and apply initial **patches**.
 - **Access Control Setup:** Implement **Authentication, Authorization, and Accounting (AAA)**, setting up user roles and permissions.
 - **Security Control Deployment:** Install and configure preventive and detective controls (e.g., firewalls, IDS/IPS, anti-malware).
 - **Acceptance Testing:** Conduct final security tests (e.g., penetration tests) to ensure the network is secure before going live.
- 4. Operations & Maintenance (O&M):** This is the longest stage, where the network is actively used. Security here is about **continuous monitoring and proactive management** to sustain a secure posture. **Key Activities:**
 - **Continuous Monitoring:** Use SIEM systems and audit logs for real-time threat detection.
 - **Vulnerability Management:** Regularly perform **vulnerability scans** and apply **patches** promptly.
 - **Configuration Management:** Monitor and enforce baseline security configurations to prevent drift.
 - **Incident Response:** Execute the **Incident Response Plan** when a security event occurs.
 - **User Training:** Conduct periodic security awareness training.
 - **Backup & Disaster Recovery:** Ensure data backups are valid and tested.

- 5. Disposal & Decommissioning:** The stage where the network components or data are retired and removed from service. Security focuses on preventing **information leakage** and ensuring physical security. **Key Activities:**
- **Data Sanitization:** Securely erase (wipe, degauss, or physically destroy) storage media to prevent data recovery (meeting the standard of **Non-repudiation** for destruction).
 - **Equipment Disposal:** Follow corporate policy for the disposal of hardware (e.g., asset tags removal, recycling programs).
 - **System De-registration:** Remove the decommissioned system from all asset inventories, network maps, AAA services, and monitoring tools.
 - **Archiving:** Retain necessary data/logs (e.g., for regulatory compliance) in a secure archive before final destruction.

Vulnerability Assessment & Penetration Testing

A **Vulnerability Assessment (VA)** is a systematic process of **identifying, quantifying, and prioritizing** the vulnerabilities in an organization's network, applications, and systems. It is primarily a broad, automated process. **Purpose**

- **Identify Weaknesses:** To discover as many security flaws as possible (misconfigurations, missing patches, outdated software).
- **Quantify Risk:** Assign severity levels (High, Medium, Low) to vulnerabilities, often using the **Common Vulnerability Scoring System (CVSS)**.
- **Compliance:** Meet regulatory requirements (e.g., PCI DSS, HIPAA) that mandate regular scanning.
- **Prioritize Remediation:** Provide a prioritized list for the IT team to fix.

Feature	Vulnerability Assessment (VA)	Penetration Testing (PT)	Risk Assessment (RA)
Goal	Identify known security weaknesses (Breadth).	Exploit weaknesses to prove access/impact (Depth).	Determine the overall business risk to assets (Strategy).
Method	Mostly Automated scanning.	Mostly Manual simulation of a real attack.	Manual and quantitative analysis of <i>Likelihood & Impact</i> .
Result	A list of unexploited flaws and their severity scores.	Proof of Concept (PoC) showing successful exploitation and business impact.	A prioritized list of risks for senior management to accept, mitigate, or transfer.
Focus	What could be broken?	Can it be broken, and what is the damage?	Why do we care about this asset/flaw?

Types of Vulnerabilities

- 1. Software Vulnerabilities:** These occur due to flaws in application code, such as buffer overflows, SQL injection, insecure APIs, or logic errors. Attackers can exploit these issues to gain unauthorized access or modify data.
- 2. Network Vulnerabilities:** These are weaknesses in network devices, protocols, or configurations — such as open ports, weak firewall rules, outdated network firmware, or insecure communication channels.

3. **Configuration Vulnerabilities:** Misconfigured systems (e.g., default passwords, unnecessary services running, improper permissions) can expose systems to attack. These are common in servers, databases, and cloud setups.
4. **Human Vulnerabilities:** These arise due to human error or lack of awareness — such as clicking phishing links, weak passwords, poor security practices, or mishandling sensitive data.

VAPT Phases

VAPT (the combination of VA and PT) follows a structured lifecycle, ensuring that testing is ethical, controlled, and effective.

1. **Planning & Scoping:**
 - **Define Goals:** What is the target (e.g., web application, internal network)? What is the objective (e.g., gain domain admin)?
 - **Define Rules of Engagement (RoE):** Crucial for legal protection; specifies start/end dates, allowed methods, contact information, and rules on sensitive data/DoS attacks.
 - **Get Authorization:** Formal written permission from asset owners (**Get-out-of-Jail-Free Card**).
2. **Information Gathering (Reconnaissance):**
 - **Passive:** Collecting publicly available information without direct interaction (e.g., using Google, social media, DNS lookups).
 - **Active:** Direct interaction with the target to map the environment (e.g., port scanning with Nmap, banner grabbing).
3. **Vulnerability Scanning:**
 - Use automated tools (e.g., Nessus) to discover technical vulnerabilities against the identified assets. This is the **Vulnerability Assessment** part of VAPT.
4. **Exploitation & Post-Exploitation:**
 - **Exploitation:** Manually (or using tools like Metasploit) attempt to leverage the discovered vulnerabilities to gain access.
 - **Post-Exploitation:** Once inside, attempt to **escalate privileges** (move from a low-level user to an administrator) and pivot to other systems to measure the true extent of the breach.
5. **Reporting & Remediation:**
 - **Reporting:** Document all steps, the **Proof of Concept (PoC)** for successful exploits, the business impact, and clear, prioritized remediation advice.
 - **Remediation:** The client fixes the vulnerabilities. The pentester often performs a final retest to **verify** that the fixes were effective.

Penetration Testing Methodologies

1. **Black-box Testing:** is a method where the tester focuses solely on the **functional requirements** of the software without any knowledge of its internal workings or code structure. The tester does not need to know how the software is designed or how it performs its tasks. Instead, the focus is on the **input-output relationships**—providing inputs and checking whether the outputs meet the expected results.
 - **Focus:** Functionality, user experience, and system behaviour.
 - **What the tester does:** Tests the software by providing inputs and verifying whether the outputs meet the functional requirements.
 - **Internal workings are irrelevant:** The internal logic, algorithms, or code structure are not examined during black-box testing.
 - **Advantages:**

- The tester doesn't need to know the software's internal code, making it easier to test from a user's perspective.
- Useful for validating high-level functionality against requirements.
- Focuses on the system's behaviour and ensures that it meets user expectations.
- **Disadvantages:**
 - May miss potential errors in the system's internal processes and logic.
 - Limited in covering all possible code paths and execution scenarios.
- **Example:** For a login system, a tester inputs different combinations of usernames and passwords to check if the system behaves correctly (i.e., accepting valid credentials and rejecting invalid ones).

2. **White-box Testing:** also known as **clear-box** or **structural testing**, involves testing the **internal structure** and **logic** of the software. The tester has full access to the code and architecture and tests the system's behaviour based on this detailed knowledge. This method helps ensure that the internal logic, code paths, and structures work as intended.

- **Focus:** Code structure, logic, algorithms, and paths.
- **What the tester does:** Tests individual parts of the system based on the code (such as functions, methods, or branches in the logic). The goal is to ensure that the code executes correctly in different scenarios, covering all possible code paths.
- **Internal workings are essential:** The tester needs a deep understanding of the code, architecture, and design of the software.
- **Advantages:**
 - Helps identify hidden errors or issues in the internal logic and structure of the software.
 - Ensures that all code paths and branches are covered, improving test coverage.
 - Effective in detecting problems related to code, algorithms, and performance.
- **Disadvantages:**
 - Requires in-depth knowledge of the code and can be time-consuming.
 - Not suitable for testing the software from an end-user's perspective.
- **Example:** For a login system, a tester would check the function handling user authentication, inspecting whether all code paths for successful and unsuccessful logins are executed correctly.

3. **Gray-box Testing:** is a hybrid approach that combines elements of both black-box and white-box testing. In gray-box testing, the tester has partial knowledge of the internal workings of the software but does not have complete access to the source code. This approach often uses a **limited understanding of the system's internals** to design tests while still focusing primarily on testing the software from the user's perspective.

- **Focus:** Combination of functionality and internal structure.
- **What the tester does:** The tester uses knowledge of the system's architecture, design, or database (but not the full code) to test the software. They might focus on areas such as security, session management, or application flows that involve internal systems.
- **Internal workings are partially known:** The tester might have access to high-level design or architecture documentation, but not the complete codebase. This allows them to create more targeted test cases than with black-box testing.
- **Advantages:**
 - More targeted than black-box testing because the tester has partial knowledge of the system.

- Combines the best of both worlds by addressing both functional aspects (like black-box) and internal structure (like white-box).
- More effective for testing system security, data handling, and integrations with other systems.
- **Disadvantages:**
 - Limited by the tester's partial knowledge of the system, which may result in missing certain defects or code issues.
 - Still requires careful design and planning to ensure proper coverage.
- **Example:** In the case of a web application, a tester might use knowledge of the backend database or server architecture to test for security vulnerabilities, such as SQL injection or data leakage, while still interacting with the application as a user would.

Testing Type	Focus	Internal Knowledge Required	Testing Approach	Example
Black-box Testing	Functionality, inputs, outputs	None	Tests software without knowledge of its code	Testing the login system with valid/invalid credentials
White-box Testing	Code structure, internal logic	Full knowledge	Tests the internal structure and logic	Testing the user authentication code for edge cases
Gray-box Testing	Combination of functionality and internal workings	Partial knowledge	Tests the system using partial understanding of the internals	Testing security vulnerabilities in a web application with partial backend knowledge